STACK ROUTE

AN NIIT VENTURE

Retail MongoDB Case Study

Retail MongoDB Case Study for Data Engineers

Raw Ingestion, Validation & Incremental Processing of Retail Transactions

1 Business Context

A large **omnichannel retail organization** operates:

- Physical stores (POS systems)
- Online platforms (web & mobile)

Each customer purchase generates a **transaction event** in JSON format. These events are pushed asynchronously from stores and online systems to the central data platform.
The retail organization faces the following challenges:

- High-volume **semi-structured transaction data**
- **Schema drift** (new fields like discounts, loyalty flags)
- **Late-arriving transactions** due to store sync delays
- Duplicate or corrected transactions
- Need to prepare clean data for **Spark-based analytics**

To address this, the organization uses **MongoDB as a Raw (Bronze) Data Store** in its data engineering architecture.

2 Problem Statement

Design and implement a **MongoDB-based raw ingestion and validation layer** that:

1. Stores retail transactions without rigid schema constraints
2. Supports identification of invalid or corrupt records
3. Handles incremental and late-arriving data
4. Enables aggregation for basic retail analytics
5. Prepares clean data for downstream Spark / SQL pipelines

3 Dataset Description

Retail Transaction Event (JSON)

```
{
  "transaction_id": "TXN1042",
  "store_id": "STR12",
  "city": "Bangalore",
  "product_category": "Electronics",
  "quantity": 1,
  "unit_price": 32000,
  "transaction_amount": 32000,
  "transaction_time": "2025-01-15T18:40:00Z",
  "payment_mode": "CARD",
  "ingested_at": "2025-01-15T18:41:10Z"
}
```


All Information within this document is Intellectual property of StackRoute (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of StackRoute (NIIT Ltd).

Key Characteristics

- Semi-structured JSON
- Append-heavy workload
- Event time (`transaction_time`) ≠ arrival time (`ingested_at`)
- Occasional:
  - Negative quantity / amount
  - Duplicate `transaction_id`
  - Late-arriving records

4 Data Engineering Objectives

Learners must demonstrate the ability to:

- Use MongoDB as a **raw ingestion layer**
- Apply **data quality checks** in NoSQL systems
- Manage **incremental loads** using arrival timestamps
- Handle **late-arriving and corrected transactions**
- Prepare data for **distributed analytics platforms**

5 MongoDB Design (Enterprise Pattern)

Database

`ness_retail_raw`

Collections

Collection	Purpose
`raw_transactions`	Raw landing of all retail events
`validated_transactions`	Cleaned & filtered records
`pipeline_control`	Stores ingestion watermark

REAL-TIME DATA FLOW (MULTIPLE BATCHES)

Think of each batch as **5–10 minutes of retail activity**.

◇ Batch 1 – Morning Store Activity (09:00–09:30)

```
db.raw_transactions.insertMany([
  {
    transaction_id: "TXN2001",
    store_id: "STR01",
    city: "Bangalore",
    product_category: "Grocery",
    quantity: 4,
    unit_price: 120,
    transaction_amount: 480,
    transaction_time: ISODate("2025-01-15T09:02:00Z"),
```


All Information within this document is Intellectual property of StackRoute (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of StackRoute (NIIT Ltd).

```
    payment_mode: "UPI",
    ingested_at: ISODate("2025-01-15T09:02:30Z")
  },
  {
    transaction_id: "TXN2002",
    store_id: "STR02",
    city: "Bangalore",
    product_category: "Electronics",
    quantity: 1,
    unit_price: 42000,
    transaction_amount: 42000,
    transaction_time: ISODate("2025-01-15T09:05:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-01-15T09:05:40Z")
  },
  {
    transaction_id: "TXN2003",
    store_id: "STR03",
    city: "Chennai",
    product_category: "Fashion",
    quantity: 2,
    unit_price: 2500,
    transaction_amount: 5000,
    transaction_time: ISODate("2025-01-15T09:10:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-01-15T09:11:10Z")
  }
])
```

◇ Batch 2 – Peak Hour Activity (09:30–10:00)

```
db.raw_transactions.insertMany([
  {
    transaction_id: "TXN2004",
    store_id: "STR04",
    city: "Hyderabad",
    product_category: "Electronics",
    quantity: 2,
    unit_price: 32000,
    transaction_amount: 64000,
    transaction_time: ISODate("2025-01-15T09:35:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-01-15T09:35:45Z")
  },
  {
    transaction_id: "TXN2005",
    store_id: "STR05",
    city: "Hyderabad",
    product_category: "Grocery",
    quantity: 6,
    unit_price: 90,
    transaction_amount: 540,
    transaction_time: ISODate("2025-01-15T09:40:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-01-15T09:40:25Z")
  },
  {
    transaction_id: "TXN2006",
    store_id: "STR06",
    city: "Chennai",
    product_category: "Electronics",
    quantity: -1,                    // ✗ Invalid
```


All Information within this document is Intellectual property of StackRoute (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of StackRoute (NIIT Ltd).

```
    unit_price: 18000,
    transaction_amount: -18000,   // ✗ Invalid
    transaction_time: ISODate("2025-01-15T09:45:00Z"),
    payment_mode: "CARD",
    ingested_at: ISODate("2025-01-15T09:46:00Z")
  }
])
```

◇ Batch 3 – Late-Arriving Store Sync (Older Event Time)

```
db.raw_transactions.insertMany([
  {
    transaction_id: "TXN1998",
    store_id: "STR02",
    city: "Bangalore",
    product_category: "Electronics",
    quantity: 1,
    unit_price: 18000,
    transaction_amount: 18000,
    transaction_time: ISODate("2025-01-15T08:50:00Z"), // OLD event time
    payment_mode: "CARD",
    ingested_at: ISODate("2025-01-15T10:05:00Z")       // Arrived late
  }
])
```

◇ Batch 4 – Duplicate & Correction Event

```
db.raw_transactions.insertMany([
  {
    transaction_id: "TXN2003",    // Duplicate ID
    store_id: "STR03",
    city: "Chennai",
    product_category: "Fashion",
    quantity: 2,
    unit_price: 2500,
    transaction_amount: 4500,     // Corrected (discount applied)
    transaction_time: ISODate("2025-01-15T09:10:00Z"),
    payment_mode: "UPI",
    ingested_at: ISODate("2025-01-15T10:10:00Z"),
    correction_flag: true
```


All Information within this document is Intellectual property of StackRoute (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of StackRoute (NIIT Ltd).